

# Manual Testcases

## Resource Consumption Testing:

1. CPU load testing, RAM/ROM consumption, and temperature testing
2. Steps, expected results, and targets
3. Registers that need to be measured
4. Methods to obtain values for CPU load, temperature, and other parameters
5. Testing algorithms
6. Tests to stress the microcontroller in order to increase CPU load and temperature

### 1. CPU load testing, RAM/ROM consumption, and temperature testing

CPU Load testing: verifici cat de mult din procesor este folosit in anumite conditii.

RAM/ROM consumption: masori cata memorie RAM si ROM este folosita.

Temperature testing: masori temperatura placii sau a microcontrollerului la diferite temperature.

### 2. Steps, expected results, and targets

Pasi: Rularea codului si testarea perifericelor.

Masurarea parametrilor.

Rezultate asteptate: CPU ul va varia in functie de complexitatea codului.

Tinte: Programul functioneaza

### 3. Registers that need to be measured

**CPU Load**: Nu exista registru dedicat, dar se poate estima prin monitorizarea contorului de instructiuni sau prin utilizarea temporizatoarelor.

**RAM/ROM**:

- RAM - Stack Pointer (SP), Heap boundaries (nu sunt registri directi, dar se pot monitoriza cu tool-uri externe).
- ROM - Dimensiunea codului compilat.

**Temperatura**: Registrul ADC (ex: ADMUX, ADCSRA) pentru citirea senzorului de temperatura intern sau extern.

#### **4. Methods to obtain values for CPU load, temperature, and other parameters**

##### **CPU Load:**

- Masurare indirecta prin temporizatoare, de exemplu masurarea timpului de executie in mod activ vs starea de repaus).
- Counters pentru instructiuni executate.

##### **Temperature:**

- Citirea senzorului intern de temperatura prin ADC.
- Citirea senzorilor externi conectati pe ADC.

##### **RAM/ROM:**

- RAM: analiza codului și utilizarea memoriei in IDE.
- ROM: dimensiunea codului.

#### **5. Testing Algorithms**

**Temperature rise:** Executarea in bucla continua fara pauze.

**Idle vs Active:** Alternarea intre strea active si de repaus.

#### **6. Tests to Stress the Microcontroller in Order to Increase CPU Load and Temperature**

- Testam microcontroller ul la temperature ridicate si scazute pentru a ii testa comportamentul.
- Activarea tuturor perifericelor simultan.
- Bucla continua cu calcule matematice.

# Afisare Text Dinamic pe Display 7 Segmente

## Descriere functionare:

### Butoane

1. **Butonul stanga (A0):**  
La fiecare apasare, se activeaza modul scroll orizontal (stanga, dreapta).  
Directia de scroll se inverseaza succesiv: stanga -> dreapta -> stanga -> dreapta, etc.
2. **Butonul dreapta (A1):**  
La fiecare apasare, se activeaza modul scroll vertical (sus, jos).  
Directia de scroll se inverseaza succesiv: sus -> jos -> sus -> jos, etc.

### Vizualizare pe display

Textul implicit este "HELO", reprezentat prin coduri binare pentru segmentele 7-segmente.

### Scroll orizontal

1. Caracterele se deplaseaza pe display de la stanga la dreapta si invers.
2. Implementat prin functiile SchimbPozitieRtoL() si SchimbPozitieLtoR().

### Scroll vertical

1. Segmentele care formeaza caracterele sunt mutate sus sau jos, creand un efect de miscare verticala
2. Implementat prin functiile shiftareBitiJos() si shiftareBitiSus().

## Am realizat teste pentru functiile de shiftare:

```
#include <Arduino.h>
```

```
#include "Teste.h"
```

```
extern byte text[][7];
```

```
extern int lungime;
```

```
extern byte textaux[][7];
```

```
extern byte textaux2[][7];
```

```
extern int lungimeaux;
```

```
bool valid=true;
```

```
void printMatrix(byte text[][7], int rows) {  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < 7; j++) {  
            Serial.print(text[i][j]);  
            Serial.print(" ");  
        }  
        Serial.println();  
    }  
}
```

```
void test_shiftareBitiRtoL() {  
    SchimbPozitieRtoL(text, lungime, 0);  
    byte expected[][7] = {  
        {1, 0, 0, 1, 0, 0, 0}, // H  
        {0, 1, 1, 0, 0, 0, 0}, // E  
        {1, 1, 1, 1, 1, 1, 1},  
        {1, 1, 1, 0, 0, 0, 1}, // L  
        {0, 0, 0, 0, 0, 0, 1}, // O  
        {1, 1, 1, 1, 1, 1, 1},  
        {1, 1, 1, 1, 1, 1, 1},  
        {1, 1, 1, 1, 1, 1, 1}  
    };  
}
```

```
/* matricea corecta pentru a nu esua testul
```

```
byte expected[][7] = {  
    {1, 0, 0, 1, 0, 0, 0}, // H  
    {0, 1, 1, 0, 0, 0, 0}, // E
```

```

{1, 1, 1, 0, 0, 0, 1}, // L
{0, 0, 0, 0, 0, 0, 1}, // O
{1, 1, 1, 1, 1, 1, 1},
{1, 1, 1, 1, 1, 1, 1},
{1, 1, 1, 1, 1, 1, 1},
{1, 1, 1, 1, 1, 1, 1}
};
*/

```

```

for (int i = 0; i < lungimeaux; i++) {
    for (int j = 0; j < 7; j++) {
        if(textaux[i][j] != expected[i][j]) {
            valid = false;
        }
    }
}

if(valid) {
    Serial.println("A mers testul test_shiftareBitiRtoL");
} else {
    Serial.println("N-a mers testul test_shiftareBitiRtoL");
    printMatrix(expected, 8);
    Serial.println();
    printMatrix(textaux, 8);
}

valid = true;
}

```

```

void test_shiftareBitiLtoR() {
    SchimbPozitieLtoR(text, lungime, 1);
}

```

```

byte expected[][7] = {
    {1, 1, 1, 1, 1, 1, 1},
    {1, 0, 0, 1, 0, 0, 0}, // H
    {0, 1, 1, 0, 0, 0, 0}, // E
    {1, 1, 1, 0, 0, 0, 1}, // L
    {0, 0, 0, 0, 0, 0, 1}, // O
    {1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1},
    {1, 1, 1, 1, 1, 1, 1}
};

```

```

for (int i = 0; i < lungimeaux; i++) {
    for (int j = 0; j < 7; j++) {
        if(textaux[i][j] != expected[i][j]) {
            valid = false;
        }
    }
}

if(valid) {
    Serial.println("A mers testul test_shiftareBitiLtoR");
} else {
    Serial.println("N-a mers testul test_shiftareBitiLtoR");
    printMatrix(expected, 8);
    Serial.println();
    printMatrix(textaux, 8);
}

valid = true;
}

```

```

void test_shiftareBitiJos() {
    shiftareBitiJos(text, lungime, 1);

    byte expected[][7] = {
        {1, 1, 0, 0, 0, 1, 1},
        {1, 1, 1, 0, 0, 1, 0},
        {1, 1, 1, 1, 0, 1, 1},
        {1, 1, 0, 1, 0, 1, 0},
    };

    bool valid = true;
    for (int i = 0; i < lungime; i++) {
        for (int j = 0; j < 7; j++) {
            if (textaux2[i][j] != expected[i][j]) {
                valid = false;
            }
        }
    }

    if (valid) {
        Serial.println("A mers testul test_shiftareBitiJos");
    } else {
        Serial.println("N-a mers testul test_shiftareBitiJos:");
        printMatrix(expected, lungime);
        Serial.println();
        printMatrix(textaux2, lungime);
    }

    valid = true;
}

```

```

void test_shiftareBitiSus() {
    shiftareBitiSus(text, lungime, 3);

    byte expected[][7] = {
        {1, 1, 1, 1, 1, 1, 1},
        {1, 1, 1, 1, 1, 1, 1},
        {1, 1, 1, 1, 1, 1, 1},
        {1, 1, 1, 1, 1, 1, 1},
    };

    bool valid = true;
    for (int i = 0; i < lungime; i++) {
        for (int j = 0; j < 7; j++) {
            if (textaux2[i][j] != expected[i][j]) {
                valid = false;
            }
        }
    }

    if (valid) {
        Serial.println("A mers testul test_shiftareBitiSus");
    } else {
        Serial.println("N-a mers testul test_shiftareBitiSus:");
        printMatrix(expected, lungime);
        Serial.println();
        printMatrix(textaux2, lungime);
    }

    valid = true;
}

```



OUTPUT PENTRU CODE COVERAGE:

```
Va da fail intentionat primul test.  
N-a mers testul test_shiftareBitiRtoL  
1 0 0 1 0 0 0  
0 1 1 0 0 0 0  
1 1 1 1 1 1 1  
1 1 1 0 0 0 1  
0 0 0 0 0 0 1  
1 1 1 1 1 1 1  
1 1 1 1 1 1 1  
1 1 1 1 1 1 1  
  
1 0 0 1 0 0 0  
0 1 1 0 0 0 0  
1 1 1 0 0 0 1  
0 0 0 0 0 0 1  
1 1 1 1 1 1 1  
1 1 1 1 1 1 1  
1 1 1 1 1 1 1  
1 1 1 1 1 1 1  
  
A mers testul test_shiftareBitiLtoR  
A mers testul test_shiftareBitiJos  
A mers testul test_shiftareBitiSus
```